KATEDRA INFORMATIKY
UNIVERZITA PALACKÉHO V OLOMOUCI
www.inf.upol.cz

Michal Krupka
krupka.inf.upol.cz
michal.krupka@upol.cz
17. listopadu 12, 771 46 Olomouc

Paradigmata programování 2 ⋄ poznámky k přednášce

2. Úvod do maker v Lispu

verze z 28. února 2025

1 Úvod

Některé speciální operátory, které jsme měli k dispozici minulý semestr, není nutné používat, protože je lze nahradit, i když méně přehledně, jinými. Jde například o operátory `let`, `let*`, `and`, `or`, `cond`.

Tady jsou příklady výrazů s těmito operátory a jejich možné náhrady:

```
                                (if a
(and a b c)    ⟶                   (if b c nil)
                                  nil)

                                (if a
(or a b c)     ⟶                   t
                                  (if b t c))

                                (if a
                                    b
(cond (a b)                       (if c
      (c d)    ⟶                      d
      (e f))                        (if e
                                        f
                                      nil)))

                                (funcall (lambda (a b)
(let ((a 1)                                (+ a b))
      (b 2))   ⟶                      1
  (+ a b))                            2)

(let* ((a 1)                    (let ((a 1))
       (b (1+ a)))   ⟶            (let ((b (1+ a)))
  (* a b))                          (* a b)))
```

Na poslední přednášce minulého semestru jsme ukázali, že této skutečnosti lze využít při konstrukci interpretu programovacího jazyka: při vyhodnocování `let`-výrazu jsme výraz převedli na jiný výraz a ten jsme znovu dali vyhodnotit. Operátor

`let` tedy v podstatě nefungoval jako speciální operátor, ale jen jako pokyn ke transformaci vyhodnocovaného výrazu v jiný. V této souvislosti jsme použili termín **makro**.

2 Makra

V původním významu slovo **makro** znamená **makroinstrukci**. Šlo o výraz v assembleru, který nebyl přímo instrukcí, ale sloužil jako zkratka pro nějakou posloupnost instrukcí. Makra se používala kvůli zestručnění kódu a také přenositelnosti, kdy na každé platformě mohlo totéž makro zastupovat jiný kód.

V dnešní době se makra běžně používají v některých formátech textových souborů jako zástupci jiného, delšího textu nebo dokonce kódu, jehož vykonáním text vznikne (jako příklad můžeme uvést typografický systém TEX nebo jazyk PHP jako vložený jazyk pro webové stránky).

V oblasti programovacích jazyků makra slouží jako pojmenování rozsáhlejších částí kódu (assemblery, C, Prolog, Swift, Lisp ...). Používají se hlavně tam, kde nestačí programová abstrakce pomocí funkcí neboli **procedurální abstrakce** (pokud si potřebujete zopakovat, co to je a jaké má výhody, podívejte se na druhou přednášku minulého semestru), a také pokud chceme upravovat a rozšiřovat přímo programovací jazyk, ve kterém pracujeme. To budou hlavní příklady používání maker v tomto semestru.

V některých moderních jazycích vycházejících z C (C++, Zig, J) se setkáváme s příbuzným principem — *Compile Time Evaluation*, který má ovšem jiný účel.

Jak uvidíme, Lisp je na používání maker vhodný díky tomu, že jeho zdrojový kód nemá textový tvar, ale je datovou strukturou tvořenou atomy a seznamy, se kterou se lépe pracuje, než s textem.

3 Jak pracují makra v Lispu

V minulém semestru byly operátory uvedené na začátku textu zavedeny jako speciální operátory, tedy operátory se zvláštním způsobem vyhodnocování. V našem jazyce pro tento semestr tyto operátory nejsou. Postupně si je definujeme sami pomocí systému maker, který je v jazyce k dispozici a který si na této a příští přednášce vysvětlíme. Systém také využijeme k definici mnoha dalších maker vlastních.

Makra v Lispu jsou zvláštní druh operátorů, které se podobají speciálním operátorům, ale na rozdíl od nich si je uživatel může sám definovat. Udělá to tak, že popíše způsob, jak se výraz s novým operátorem převede na jiný výraz, jehož operátor je Lispu znám.

Ukážeme si to na příkladu. Řekněme, že chceme definovat operátor `and-2`, který realizuje logickou konjunkci, pro jednoduchost zatím jen pro dva argumenty. Důvod, proč to chceme udělat, je jasný. I když bychom se bez takového operátoru mohli

obejít tím, že bychom místo něj používali operátor `if`, mít jej k dispozici umožní zjednodušit nám práci a zvýšit přehlednost programu.

Operátor `and-2` nechceme definovat jako funkci, protože tak by neumožňoval používat zkrácené vyhodnocování argumentů (které jsme v minulém semestru mnohokrát využili například, když jsme psali rekurzivní funkce deklarativním způsobem).

Napíšeme jej jako **makro**. Řekneme Lispu, že kdykoli narazí na takovýto výraz:

```
(and-2 a b)
```

má jej chápat jako výraz

```
(if a b nil)
```

který už bude umět vyhodnotit. (Podvýrazy ***a*** a ***b*** píšu kurzívou, protože nejde o proměnné `a`, `b`, ale o zástupce za libovolné výrazy.) Tomuto kroku se říká **expanze makra**.

Co to makro je, není blíže specifikováno, ale je řečeno, že každé makro je dáno dvěma údaji:

1. Symbolem, kterému říkáme **název** makra a který makro zastupuje ve zdrojovém kódu jako operátor složených výrazů (v našem příkladě je to symbol `and-2`). Pokud je symbol názvem makra, nemůže být názvem funkce ani speciálním operátorem.

2. Přesně definovaným způsobem, kterým výraz, jehož operátorem je název makra, expanduje na jiný výraz. Jak uvidíme za chvíli, tento způsob je popsán uživatelskou funkcí Lispu.

Výraz, jehož operátorem je název makra, se nazývá **makro-výraz**.

Vyhodnocení makro-výrazu probíhá ve dvou krocích:

1. výraz se expanduje,

2. výsledek expanze se znovu vyhodnotí.

Úplný vyhodnocovací proces v Lispu (včetně maker) tedy pracuje takto:

Vyhodnocení výrazu E v prostředí *env*

Je-li E symbol, výsledkem je hodnota symbolu E v prostředí *env*.

Je-li E jiný atom než symbol, výsledkem je E.

Je-li E seznam s operátorem o a argumenty $a_1 \ldots a_n$, pak o musí být symbol a:

Jestliže o je speciální operátor, seznam se vyhodnotí podle pravidel tohoto operátoru.

Jestliže o je název makra, E se expanduje podle pravidel pro toto makro a výsledný výraz se vyhodnotí v prostředí *env*.

Jinak

1. se zjistí hodnota f funkční vazby symbolu o v prostředí *env* (je to funkce),
2. zjistí se hodnoty $v_1 \dots v_n$ argumentů $a_1 \dots a_n$ v prostředí *env* (opět vyhodnocovacím procesem).
3. Výsledkem je výsledek aplikace funkce f na hodnoty $v_1 \dots v_n$. Aplikace funkce může mít též vedlejší efekt. Funkce mohou vracet více hodnot.

4 Jak se definují makra

Napsat makro znamená popsat způsob, jakým se složené výrazy, jejichž operátorem je název makra, expandují. Zde můžeme těžit z toho, že složené výrazy jsou seznamy a expanzi popsat jako funkci v Lispu. Argumenty takové funkce jsou (nevyhodnocené!) argumenty expandovaného výrazu. Například expanze makra `and-2` může být vykonaná touto funkcí:

```
(defun and-2-expander (a b)
  (list 'if a b nil))
```

Funkci, která vykonává expanzi makra říkáme**expanzní funkce makra** Definovat makro znamená zadat jeho název a expanzní funkci. To se dělá pomocí speciálního operátoru `defmac`. Například makro `and-2` můžeme definovat takto:

```
(defmac and-2 #'and-2-expander)
```

a podívat se, jak funguje:

```
> (and-2 (< 1 2) (plusp 0))
NIL

> (and-2 (< 1 2) (plusp 1))
T

> (and-2 (> 1 2) (plusp 1))
NIL
```

Po napsání makra bychom ale především měli otestovat, že správně expanduje, tj. otestovat jeho expanzní funkci. Tu můžeme kdykoli zjistit z názvu makra pomocí vestavěné funkce `macro-function`:

```
> (macro-function 'and-2)
#<FUNCTION AND-2-EXPANDER>
```

Test expanze makra tedy může vypadat takto:

```
> (funcall (macro-function 'and-2) '(< 1 2) '(plusp 0))
(IF (< 1 2) (PLUSP 0) NIL)
```

Aby se nám expanze maker testovaly pohodlněji, napíšeme si funkci `macroexpand-1`, která bude umět expandovat přímo zadaný makro-výraz. Výrazy, které nejsou makro-výrazy, by měla vracet nezměněné:

```
> (macroexpand-1 '(and-2 (< 1 2) (plusp 0)))
(IF (< 1 2) (PLUSP 0) NIL)

> (macroexpand-1 '(+ 1 2))
(+ 1 2)

> (macroexpand-1 12)
12
```

Napsat její definici pomocí funkce `macro-function` není obtížné, jen se musíme vypořádat s tím, že nemáme k dispozici operátor `let` (což brzy napravíme):

```
(defun macroexpand-1 (form)
  (labels ((me (car cdr)
             (if (macro-function car)
                 (apply (macro-function car) cdr)
               form)))
    (if (consp form)
        (me (car form) (cdr form))
      form)))
```

Obtížnost psaní maker

Při psaní maker děláme dvě nesouvisející věci zároveň:

1. vymýšlíme, co má být expanzí daného makro-výrazu (a v tom můžeme udělat chybu),

2. píšeme funkci, která výraz na expanzi převádí (a v tom můžeme udělat chybu).

Proto je důležité tyto dva kroky oddělit.

Práce s makry je práce na úrovni programovacího jazyka. Dá se zhruba říci, že definicí nového makra upravujeme programovací jazyk. Makro můžeme chápat jako **uživatelsky definovaný speciální operátor**. Expanzní funkce makra pracuje zejména se seznamy (transformuje symbolické výrazy na symbolické výrazy).

5 Zpětný apostrof

Aby byl z definice makra zřejmější tvar výsledného výrazu, můžeme použít syntax se zpětným apostrofem (*backquote*). Například místo

```
(list 'if a b 'nil)
```

můžeme napsat

```
`(if ,a ,b nil)
```

Zpětný apostrof funguje podobně jako klasický apostrof, tj. potlačí vyhodnocování výrazu za ním. Vyhodnocení podvýrazů složených výrazů lze pak zařídit čárkou. Další příklady:

```
> `((+ 1 1) ,(+ 1 1))
((+ 1 1) 2)

> `((+ 1 (+ 1 1)) (+ 1 ,(+ 1 1)) ,(+ 1 (+ 1 1)))
((+ 1 (+ 1 1)) (+ 1 2) 3)
```

Čárku lze také použít v kombinaci se zavináčem. Dosáhneme tak rozpuštění seznamu vzniklého vyhodnocením podvýrazu:

```
> `((list 1 2) ,(list 1 2) ,@(list 1 2))
((LIST 1 2) (1 2) 1 2)
```

Výraz `` `((list 1 2) ,(list 1 2) ,@(list 1 2)) `` je ekvivalentní např. výrazu `(list (list 'list 1 2) (list 1 2) 1 2)`.

Pokud by v proměnné `rest` byl uložen seznam `(c d)`, pak dostaneme:

```
> `(a b ,@rest e)
(A B C D E)
```

Výraz je ekvivalentní např. výrazu `(append (list 'a 'b) rest (list 'e))`.

Příklad

Napíšeme makro `when`, které nejprve otestuje danou podmínku a pokud je splněna, vyhodnotí postupně zadané výrazy a výsledek posledního vrátí. Není-li splněna, vrátí jen `nil`:

```
> (when (> 1 0) 'ano)
ANO

> (when (> 1 0) (out "Splněno") 'ano)
Splněno
ANO

> (when (< 1 0) (out "Splněno") 'ano)
NIL
```

Makro můžeme napsat takto:

```
(defmac when
        (lambda (test &rest expressions)
          `(if ,test (progn ,@expressions) nil)))
```

Test expanze:

```
> (macroexpand-1 '(when (> 1 0) (print "Tisk") 'ano))
(IF (> 1 0) (PROGN (PRINT "Tisk") (QUOTE ANO)) NIL)
```

Příklad

A definice operátoru `let` jako makra:

```
(defmac let
        (lambda (bindings &rest body)
          `(funcall (lambda ,(mapcar #'car bindings)
                      ,@body)
                    ,@(mapcar #'cdr bindings))))
```

Makro otestujte (expanzi i výsledek) sami.

Následující makro `defmacro` zjednodušuje definici maker tím, že s jeho pomocí stačí zadávat λ-seznam a tělo expanzní funkce. Makro `let` pak bude stačit definovat takto:

```
(defmacro let (bindings &rest body)
  `(funcall (lambda ,(mapcar #'car bindings)
              ,@body)
            ,@(mapcar #'cdr bindings)))
```

Makro `defmacro` je napsáno takto:

```
(defmac defmacro
        (lambda (name lambda-list &rest body)
          `(defmac ,name (lambda ,lambda-list ,@body))))
```

Nové symboly v jazyce

Vestavěné

speciální operátor: `defmac`

funkce: `macro-function`

Definované na přednášce

funkce: `macroexpand-1`

makra: `let`, `when`, `defmacro`

Definované na cvičení

makra: `unless`, `whenb`

Otázky a úkoly na cvičení

Makra implementujte vždy nejprve bez použití zpětného apostrofu a teprve potom s ním. Při testování vždy nejprve vyzkoušejte expanzi.

1. Co je to makro? Jak probíhá vyhodnocovací proces s makry?

2. Definujte makro `or-2` na výpočet logické disjunkce pro dva argumenty se zkráceným vyhodnocováním. Makro by mělo expandovat, jak je naznačeno na začátku textu.

3. Makro `unless` by mělo pracovat stejně jako makro `when`, ale výrazy by vyhodnocovalo, když daná podmínka není splněna:

```
> (unless (> 1 0) 'ano)
NIL

> (unless (> 1 0) (out "Nesplněno") 'ne)
NIL

> (unless (< 1 0) (out "Nesplněno") 'ne)
Nesplněno
NE
```

   Makro definujte dvěma způsoby: nejprve tak, že použijete přímo speciální operátor `if` a tělo dáte do druhé větve, a potom pomocí makra `when` s použitím funkce `not`. *(do jazyka)*

4. Napište makro `if-zero`, které má stejnou syntax jako operátor `if`, ale místo podmínky očekává číslo. Pokud je číslo nula, vyhodnotí se první větev, jinak se vyhodnotí druhá:

```
> (if-zero (+ 1 1) (+ 1 2) (+ 1 3))
4

> (if-zero (- 1 1) (- 1 2) (- 1 3))
-1
```

5. Napište následující verzi makra `when`: Makro `whenb` akceptuje na rozdíl od makra `when` jako svůj první argument dvouprvkový seznam obsahující symbol a podmínku. Na symbol bude v těle makra navázán výsledek vyhodnocení podmínky. Výrazy těla se vyhodnotí, pokud hodnota podmínky bude *Pravda*. Příklad:

```
> (bind l (list 1 2 nil))
(1 2 NIL)

> (whenb (x (second l)) (+ x 1))
3

> (whenb (x (third l)) (+ x 1))
NIL
```

V definici makra můžete použít makro `when`. *(do jazyka)*

6. Napište makro `reverse-progn`, které pracuje stejně jako speciální operátor `progn`, ale výrazy vyhodnocuje v opačném pořadí:

```
> (reverse-progn 1 (out 2) (out 3) (out 4))
4
3
2
1
```

Na obrácení seznamu můžete použít funkci `reverse`.

7. Napište makro `bind-list` na vytvoření nového prostředí, jehož symboly jsou uvedeny jako první argument a hodnoty v seznamu jako hodnota druhého argumentu. Makro má dále tělo, jehož výrazy se v novém prostředí vyhodnotí a hodnota posledního vrátí jako výsledek. Příklad:

```
> (bind-list (a b c) (list 1 (+ 1 1) (- 1 1))
             (out "a: " a ", b: " b ", c: " c)
             (+ a b c))
a: 1, b: 2, c: 0
3
```

(do jazyka)

8. Jak víme z obecného popisu vyhodnocovacího procesu, každý atom, který není symbolem, se vyhodnocuje sám na sebe. Ověřte tuto skutečnost co nejvíce způsoby pro funkce.